

Given below is the sample program that shows the construction of custom directives.

```
<html>
<head>
<title> Building Your Own Directives</title>
</head>
<body>
<h2>AngularJS Test Application</h2>
<div ng-app = "mainApp" ng-controller =
"testEmployeeController">
  <Employee name = "ABC"></Employee><br/>
  <Employee name = "XYZ"></Employee>
</div>
<script src = "http://ajax.googleapis.com/ajax/libs/
angularjs/1.3.14/angular.min.js"></script>
<script>
  var mainApp = angular.module("mainApp", []);
  mainApp.directive('Employee', function() {
    var directive = {};
    directive.restrict = 'E';
    directive.template = "Employee: <b>{{Employee.
name}}</b>, ID: <b>{{Employee.id}}</b>";
    directive.scope = {
      Employee : "=name" }
    directive.compile = function(element, attributes)
{
      element.css("border", "1px solid #cccccc");
      var testFunction = function($scope, element,
attributes) {
        element.html("Employee: <b>"+$scope.Employee.
name +"</b>, ID: <b>"+ $scope.Employee.id + "</b><br/>");
        element.css("background-color", "#00FFFF
");
      }
      return linkFunction;
    }
    return directive;
  });
  mainApp.controller('TestEmployeeController',
function($scope) {
    $scope.ABC = {};
    $scope.ABC.name = "ABC";
    $scope.ABC.id = 1;
    $scope.XYZ = {};
    $scope.XYZ.name = "XYZ";
    $scope.XYZ.id = 2;
  });
</script>
</body>
</html>
```

The output is the AngularJS test application:

Employee: ABC, ID: 1
Employee: XYZ, ID: 2

Dependency injection

AngularJS contains a built-in dependency injection mechanism. This is the key to making easily reusable and testable components. It is a software pattern that ensures references to other components are not created directly for the components. Instead of direct instantiation, each component will get references required by other components. So there is no need to find the dependencies for components, as dependencies themselves are configurable. The application will be divided into different components, which can be injected into each other. This makes your application easily reusable, configurable and testable.

AngularJS contains the following core types of objects and components:

- Value
- Factory
- Service
- Provider
- Constant

Value

Value is a simple object and used to inject values to the controller, factories or services during the configuration phase. It can be a number, string or JavaScript object. A value must belong to an AngularJS module.

Here are some examples that add values to an AngularJS module:

```
var myModule = angular.module("testModule", []);
testModule.value("numberValue", 100);
testModule.value("stringValue", "ABC");
testModule.value("objectValue", { val1 : 100, val2 : "XYZ"
});
```

The values are defined using the *value()* function on the module. The first parameter is the name of the value, and the second parameter is the value itself. Factories, services and controllers can now refer to these values by their name.

Factory

Factory is a function that is used to create value. It creates value whenever a service or controller needs it. The *factory* function is used to calculate and return the value. Once it is created, the value is reusable for all services, controllers, etc.

Here is an example that defines a factory on a module, and a controller which gets the factory-created value injected:

```
var testModule = angular.module("testModule", []);
testModule.factory("testFactory", function() { return
```